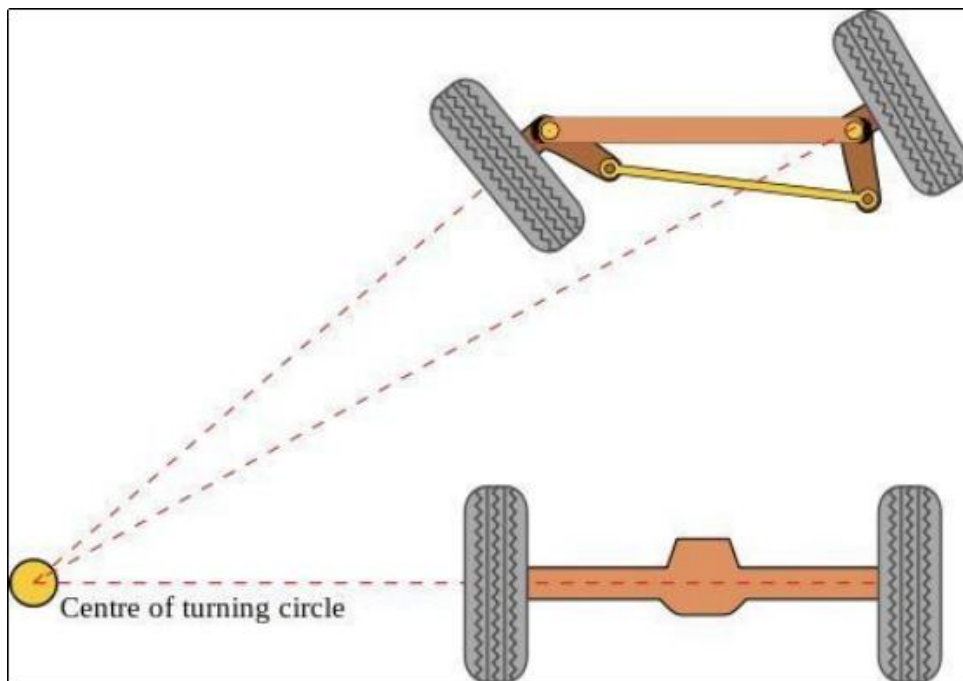


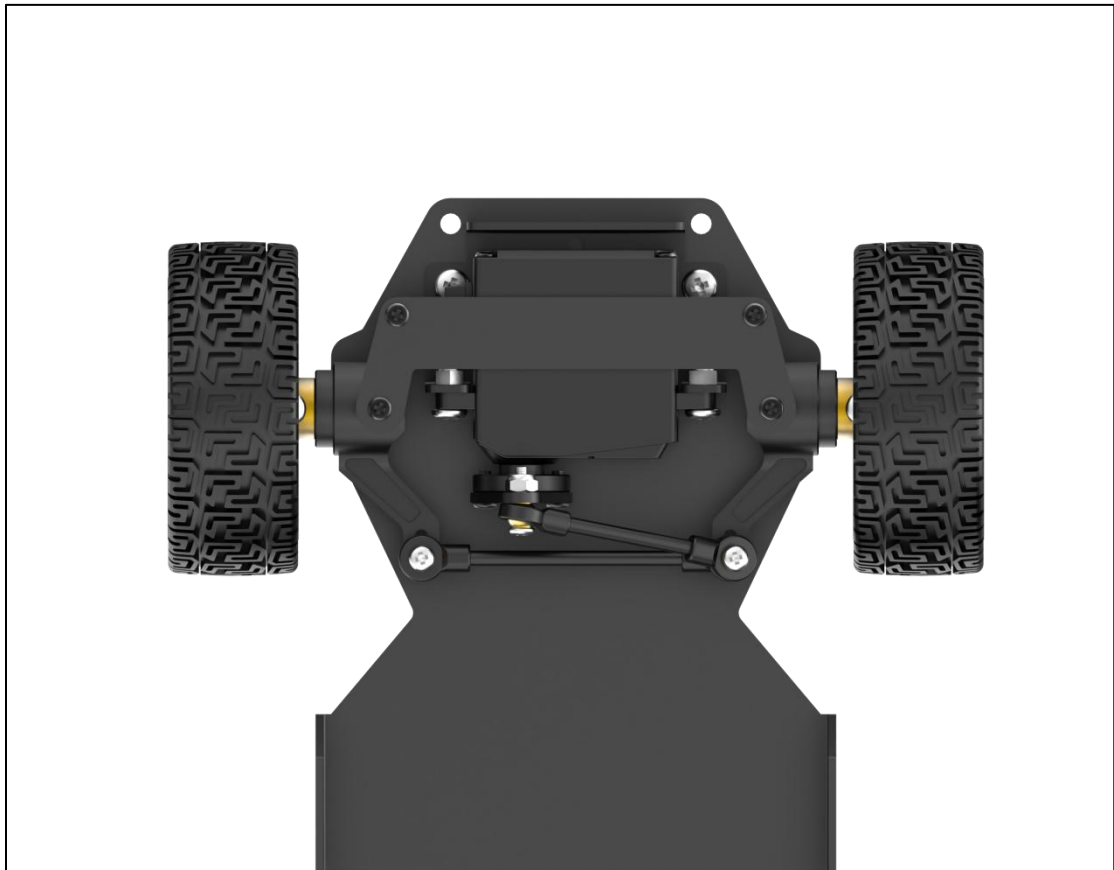
Ackerman Chassis Motion Analysis

1. Hardware Structure

The transmission mechanism of the Ackermann chassis front wheels consists of a servo, linkage, and wheels. The servo is connected to the linkage, which, in turn, is connected to the wheels. The rotation of the servo controls the degree of rotation of the linkage, thereby influencing the steering of the front wheels.

During a turn, the front two wheels remain parallel to each other, both rotating at the same angle. The rear wheels are controlled by the motor and wheels, with the motor's rotation determining the robot's forward, backward, and speed movements.





2. Physical Characteristics

The design objective of the Ackermann chassis is to optimize steering performance and stability. This is achieved through a principle known as "Ackermann geometry," which governs the difference in steering angles between the front and rear wheels. By allowing the inner front wheel to have a greater steering angle, the Ackermann chassis facilitates easier control during turns and reduces the likelihood of sliding while steering.

Additionally, the Ackermann chassis incorporates a well-designed suspension system, which plays a critical role in connecting the wheels to the vehicle body. The suspension system significantly affects the physical characteristics of the chassis and is typically independent for each wheel. This independent suspension enhances performance, contributing to better stability and a smoother ride.

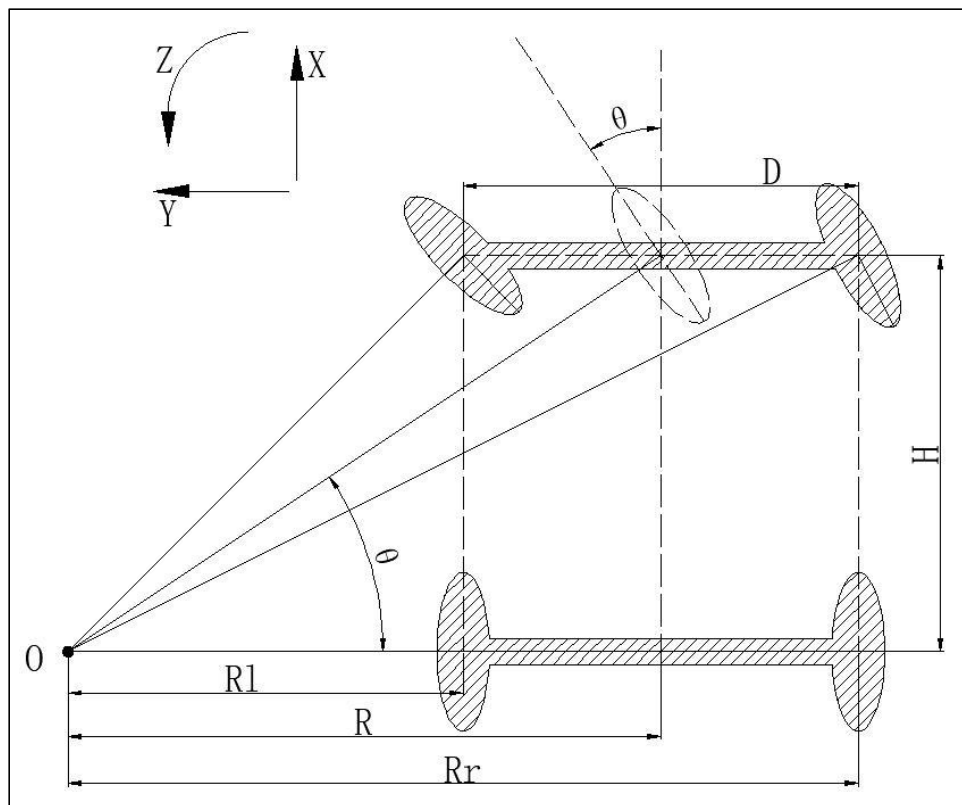
The chassis also takes into account the vehicle's center of gravity, as its position greatly influences stability and handling. Generally, the Ackermann chassis positions the center of gravity lower to minimize the risk of tilting or sliding.

Finally, the physical characteristics of the Ackermann chassis include the braking and power transmission systems. The braking system ensures stable performance by controlling the braking force on the wheels, while the power transmission system is responsible for transferring engine power to the wheels, directly impacting acceleration and driving performance.

3. Motion Principle and Formula

When conducting kinematic analysis of the Ackermann chassis, we can use the following mathematical formulas and parameters to describe its motion characteristics:

To achieve pure rolling motion for the Ackermann vehicle—meaning no side slip during turns—it is essential to ensure that the normals of the motion directions of all four wheels (the lines perpendicular to the direction of tire rolling) intersect at a single point. This point is known as the center of rotation. To simplify the model, we assume that the front axle has a single wheel, which serves as a theoretical representation of the front wheels. This simplification is depicted by the dashed line in the diagram.



1. **Front Wheel Steering Angle (θ):** The rotation angle of the front wheels, indicating the angle by which the front wheels deviate from the vehicle's forward direction. It is typically measured in radians (rad).
2. **Vehicle Linear Velocity (V):** The overall linear speed of the vehicle, representing its translational velocity. It is usually measured in meters per second (m/s). The left rear wheel speed is denoted as (V_L), and the right rear wheel speed is denoted as (V_R).
3. **Vehicle Track Width (D):** The distance between the wheels on the left and right sides of the vehicle, measured in meters (m).
4. **Wheelbase of the Vehicle (H):** The distance between the front and rear wheels of the vehicle, measured in meters (m)
5. **Vehicle Turning Radius (R):** The radius of the circle described by the vehicle during a turn, measured in meters (m). The turning radius for the left wheel is (R_L), and for the right wheel is (R_R).

Process for Calculating Robot Speed and Angle:

6. Consistency of angular velocity:

$$\omega = \frac{V}{R} = \frac{V_L}{R_L} = \frac{V_R}{R_R}$$

In this context:

ω represents the angular velocity of the vehicle.

R denotes the turning radius of the vehicle.

V is the linear velocity of the vehicle.

VL is the linear velocity of the left rear wheel.

VR is the linear velocity of the right rear wheel.

RL is the turning radius of the left wheel.

RR is the turning radius of the right wheel.

7. The relationship between the front wheel steering angle and the turning radius of the vehicle:

$$\tan \theta = \frac{H}{R} \quad , \quad R_L = R - \frac{D}{2} \quad , \quad R_R = R + \frac{D}{2}$$

H represents the distance between the front and rear wheels of the vehicle.

R signifies the turning radius of the vehicle.

D denotes the distance between the wheels on the left and right sides of the vehicle.

θ indicates the steering angle of the front wheels.

8. The speeds of the left and right wheels of the robot can be determined as:

$$V_L = \frac{V}{R} R_L = V * \left(1 - \frac{D * \tan \theta}{2H} \right)$$

9. The speeds of the left and right wheels of the robot can be determined as:

$$V_R = \frac{V}{R} R_R = V * \left(1 + \frac{D * \tan \theta}{2H} \right)$$

By knowing the wheelbase, track width, robot speed, and the steering angle of the servo, it is possible to calculate the speeds of the two rear wheels of the robot.

4. Program Outcome

The program files are located in:

ros2_ws\src\driver\controller\controller\ackermann.py

```
class AckermannChassis:
    # wheelbase = 0.213 # 前后轴距(distance between front and rear axles)
    # track_width = 0.222 # 左右轴距(distance between left and right axles)
    # wheel_diameter = 0.101 # 轮子直径(wheel diameter)

    def __init__(self, wheelbase=0.145, track_width=0.133, wheel_diameter=0.067):
        self.wheelbase = wheelbase
        self.track_width = track_width
        self.wheel_diameter = wheel_diameter
```

Ackermann wheel kinematics module, used to calculate wheel speeds and implement Ackermann wheel kinematics.

Init:

```
12 def __init__(self, wheelbase=0.145, track_width=0.133, wheel_diameter=0.067):
13     self.wheelbase = wheelbase
14     self.track_width = track_width
15     self.wheel_diameter = wheel_diameter
```

Initialize the wheel dimensions for easier subsequent calculations

speed_covert:

```
17 def speed_covert(self, speed):
18     """
19     covert speed m/s to rps/s
20     :param speed:
21     :return:
22     """
23     return speed / (math.pi * self.wheel_diameter)
```

Convert m/s to rps/s based on the wheel parameters.

set_velocity:

```

25 def set_velocity(self, linear_speed, angular_speed, reset_servo=True):
26     servo_angle = 1500
27     data = []
28     if abs(linear_speed) >= 1e-8:
29         if abs(angular_speed) >= 1e-8:
30             theta = math.atan(self.wheelbase*angular_speed/linear_speed)
31             steering_angle = theta
32             # print(math.degrees(steering_angle))
33             if abs(steering_angle) > math.radians(37):
34                 for i in range(4):
35                     msg = MotorState()
36                     msg.id = i + 1
37                     msg.rps = 0.0
38                     data.append(msg)
39                 return None, data
40             servo_angle = 1500 + int(1000*math.degrees(steering_angle)/180)
41
42             vr = linear_speed + angular_speed*self.track_width/2
43             vl = linear_speed - angular_speed*self.track_width/2
44             v_s = [self.speed_convert(v) for v in [0, vl, 0, -vr]]
45             for i in range(len(v_s)):
46                 msg = MotorState()
47                 msg.id = i + 1
48                 msg.rps = float(v_s[i])
49                 data.append(msg)
50             return servo_angle, data
51         else:
52             for i in range(4):
53                 msg = MotorState()
54                 msg.id = i + 1
55                 msg.rps = 0.0
56                 data.append(msg)
57             return None, data

```

Based on the input speed parameters, decompose them, calculate the speeds using speed_convert, and then publish the calculated radian speeds to the motors. Calculate the required steering angle from the linear and angular velocities, convert it, and send it to the servos.